Python Programming

Complete Study Guide for Units I - IV

Clear explanations, tables and real code with real outputs - every topic of the syllabus

Unit	Topics covered	Section
I	Algorithms, pseudocode, flowcharts, computational thinking; beginning with Python; modules, statements and expressions, whitespace, comments; tokens, naming objects, variables, objects and types; operators (arithmetic, relational, Boolean) and precedence	1.1 - 1.11
II	Conditional statements: if, if-else, elif ladder, nested if; repetition: for, while, nested loops, break/continue/loop-else	2.1 - 2.9
III	Functions: definition, flow of control, parameters, arguments, default values, scope, lambda, functions as objects; strings: type, operations, methods, formatted output	3.1 - 3.11
IV	Lists (iteration, indexing, slicing, operators, lists vs strings, split, anagrams), tuples from lists; dictionaries (indexing, assignment, methods); sets (methods, operators, functions)	4.1 - 4.13

How to use this guide

- Read each section, then **type the code yourself** and change the values. The green box (or the box below the code) shows the exact output.
- Blue boxes (**Remember**) are exam points. Yellow boxes (**Note**) are common mistakes.
- Colour code: grey = code, green = output. Use the Python interpreter or any online editor to practise.

Syllabus Checklist: where to find each topic

Syllabus topic	Section
Flowcharts, algorithms, pseudocode, computational thinking	1.1 - 1.4
Beginning with Python; modules, statements and expressions, whitespace, comments	1.5 - 1.6
Tokens, naming objects, variables, objects and types	1.7 - 1.9
Operators, relational operators, Boolean operators, precedence	1.10 - 1.11
Selection: if, else-if, nested statements	2.1 - 2.5
Repetition: for, while, nested loops	2.6 - 2.10
Functions: parameters, arguments, default values, flow of control, scope	3.1 - 3.5
Lambda functions, functions as objects	3.6 - 3.7
Strings: type, operations, methods, formatted output	3.8 - 3.11
Lists: indexing, slicing, iteration, operators, lists vs strings, split, anagrams	4.1 - 4.7
Tuples, tuples from lists	4.8
Dictionaries: indexing, assignment, methods	4.9 - 4.10
Sets: methods, operators and functions	4.11 - 4.13
Solved programs and practice questions for every unit	1.13, 2.10, 3.12, 4.14, last page

UNIT I — Basics of Computer and Programming

Algorithms, flowcharts, pseudocode, computational thinking and Python fundamentals

1.1 Algorithm

An **algorithm** is a finite, step-by-step set of instructions that solves a problem. Think of a recipe: the same steps in the same order always give the same dish. Before writing any program, we first design the algorithm.

- **Input**: zero or more values are given.
- **Output**: at least one result is produced.
- **Definiteness**: every step is clear and unambiguous.
- **Finiteness**: it must end after a limited number of steps.
- **Effectiveness**: every step is simple enough to be carried out.

Example - algorithm to find the largest of three numbers A, B, C:

- Step 1: Start.
- Step 2: Read A, B, C.
- Step 3: If A is greater than B and A is greater than C, largest = A.
- Step 4: Else if B is greater than C, largest = B. Step 5: Else largest = C.
- Step 6: Print largest. Step 7: Stop.

1.2 Pseudocode

Pseudocode is an informal way of writing an algorithm using plain English mixed with programming-like keywords. It is not tied to any language, so it cannot be run; it only helps us plan. Common keywords: START, END, READ/INPUT, PRINT/DISPLAY, IF-THEN-ELSE-ENDIF, WHILE-ENDWHILE, FOR-ENDFOR, SET (assignment).

```
START
  READ n
  SET sum = 0
  FOR i = 1 TO n
      sum = sum + i
  ENDFOR
  PRINT sum
END
```

The same pseudocode written in Python:

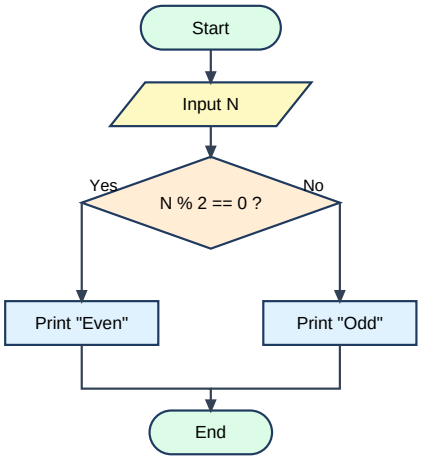
```
n = 5
total = 0
for i in range(1, n + 1):
    total = total + i
print(total)
```

```
15
```

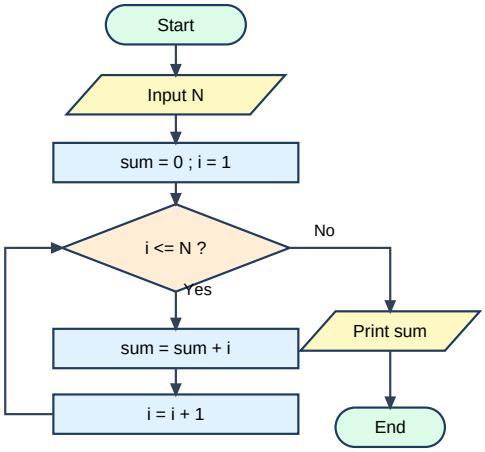
1.3 Flowchart

A **flowchart** is a diagram that shows the flow of an algorithm using standard symbols joined by arrows. It is read from top to bottom.

Symbol	Name	Used for
Rounded box (oval)	Terminator	Start and End of the program
Parallelogram	Input / Output	Reading input, printing output
Rectangle	Process	Calculations and assignments
Diamond	Decision	Yes/No question (if, while condition) with two exits
Arrow	Flow line	Direction of flow
Small circle	Connector	Joins parts of a large flowchart



(a) Decision: even or odd



(b) Loop: sum of 1 to N

1.4 Computational Thinking

Computational thinking is a way of solving problems like a computer scientist. It has four pillars:

Pillar	Meaning	Example: "Find the topper of a class"
Decomposition	Break a big problem into small parts	Read marks, compare marks, remember the best, print the result
Pattern recognition	Spot similarities and repeated steps	Comparing one student is the same job repeated for every student
Abstraction	Ignore unimportant details	Only name and marks matter, not height or hobbies
Algorithm design	Write clear steps in the right order	Start with the first student as best, update when a higher mark is found

```
marks = {"Riya": 78, "Aman": 91, "Neha": 85}
best_name, best_mark = "", -1
for name, m in marks.items():      # the repeated step
    if m > best_mark:              # the comparison
        best_name, best_mark = name, m
print("Topper:", best_name, best_mark)
```

```
Topper: Aman 91
```

1.5 Beginning with Python

Python is a **high-level, interpreted, general-purpose** language created by Guido van Rossum (1991). Its main strengths: simple readable syntax, dynamic typing, a huge standard library, and portability (runs on Windows, Linux, macOS). It is used for web development, data science, AI, automation and more.

- **Interpreted**: code is executed line by line by the Python interpreter (no separate compile step for you).
- **Dynamically typed**: you do not declare types; the type is decided from the value.
- **Two ways to run**: *interactive mode* (type commands at the >>> prompt) and *script mode* (write a .py file and run it with python file.py).
- **Case-sensitive**: Total, total and TOTAL are three different names.

Your first program uses two basic functions: `print()` shows output and `input()` reads input (always as a string).

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Hello", name)
print("Next year you will be", age + 1)
```

```
Enter your name: Riya
Enter your age: 19
Hello Riya
Next year you will be 20
```

`print()` has useful options: `sep` changes the separator between values (default space) and `end` changes what is printed at the end (default newline).

```
print("a", "b", "c", sep="-")
print("Hello", end=" ")
print("World")
print("Python", 3.12, True)
```

```
a-b-c
Hello World
Python 3.12 True
```

1.6 Parts of a Python Program

Modules. A module is a file of ready-made code (functions, constants) that you bring into your program with `import`. Python ships with many modules in its *standard library*.

```
import math
from random import randint
import datetime as dt

print(math.sqrt(49), math.pi)
print(math.floor(4.7), math.ceil(4.2), math.pow(2, 5))
print(1 <= randint(1, 6) <= 6)
print(dt.date(2026, 11, 13).strftime("%d-%m-%Y"))
```

```
7.0 3.141592653589793
4 5 32.0
True
13-11-2026
```

- `import math` imports the whole module; use `math.sqrt()`.
- `from math import sqrt` imports one name directly.
- `import numpy as np` gives the module a short nickname.
- You can create your own module: save functions in `mytools.py` and write `import mytools`.
- Use `dir(math)` to see what a module provides, and `help(math.sqrt)` for help.

Statements and expressions. An **expression** produces a value (`2 + 3`, `x * 5`, `len(s)`). A **statement** performs an action (assignment, `x = 5`, `print(x)`, `if ...:`). Every expression can appear inside a statement, but not the other way round.

```
x = 10              # assignment statement
y = x * 2 + 1       # expression 'x * 2 + 1' inside a statement
print(y)            # print statement with the expression y
a = b = 0           # chained assignment
m, n = 3, 4         # multiple assignment
m, n = n, m         # swap using tuple packing
print(m, n)
```

```
21
4 3
```

Whitespace and indentation. Python uses **indentation** (spaces at the start of a line) to mark blocks of code - there are no braces. All lines in a block must be indented the same amount (4 spaces is the convention). Blank lines and spaces around operators are ignored but make code readable. A long line can be continued with a backslash or inside brackets.

```
if 5 > 3:
    print("block line 1")
    print("block line 2")
print("outside the block")
total = 1 + 2 + \
        3 + 4
values = [1, 2,
          3, 4]
print(total, values)
```

```
block line 1
block line 2
outside the block
10 [1, 2, 3, 4]
```

Note: Wrong or mixed indentation gives an `IndentationError`. Never mix tabs and spaces.

Comments. A comment is a note for humans that Python ignores. Single-line comments start with #. For several lines, use consecutive # lines or a triple-quoted string. A triple-quoted string placed first inside a function is a **docstring** (used for documentation).

```
# This is a single-line comment
x = 5   # comment after code
"""This is a multi-line
note, often used as a docstring"""
print(x)
```

```
5
```

1.7 Special Python Elements: Tokens

A **token** is the smallest meaningful unit of a Python program. Python source code is broken into these five kinds of tokens:

Token type	Meaning	Examples
Keywords	Reserved words with fixed meaning	if, else, for, while, def, return, True, None, and, or, not
Identifiers	Names given to variables, functions, classes	total, student_name, calc_area
Literals	Fixed constant values	10, 3.14, "hello", True, None
Operators	Symbols that perform operations	+, -, *, /, ==, and

Token type	Meaning	Examples
Punctuators / Delimiters	Structure symbols	( ) [ ] { } , : . = @

```
import keyword
kw = keyword.kwlist
print(len(kw), "keywords:")
for i in range(0, len(kw), 9):
    print(*kw[i:i + 9])
```

```
35 keywords:
False None True and as assert async await break
class continue def del elif else except finally for
from global if import in is lambda nonlocal not
or pass raise return try while with yield
```

1.8 Naming Objects (Identifiers)

- Can contain letters, digits and underscore _; must **not start with a digit**.
- No spaces or special symbols (`@ # $ % -` etc.).
- Cannot be a keyword (`class = 5` is illegal).
- Case-sensitive. Choose meaningful names: `total_marks` is better than `tm`.
- Convention (PEP 8): variables and functions in `snake_case`, constants in `UPPER_CASE`, classes in `CamelCase`.

```
student_name = "Riya"     # valid
_count = 3                # valid
marks2 = 90               # valid
PI = 3.14159              # constant by convention
# 2marks = 5   -> invalid: starts with a digit
# my-name = 1  -> invalid: hyphen not allowed
print(student_name.isidentifier(), "2marks".isidentifier())
```

```
True False
```

1.9 Variables, Objects and Types

A **variable** is a name that refers to an **object** stored in memory. Every object has a **type** (what kind of data), a **value** and an **identity** (its memory id). Assigning with = makes the name point to an object. Python figures out the type automatically (**dynamic typing**), and the same name can later point to a different type.

Type	Meaning	Example
int	whole numbers (any size)	42, -7, 10**20
float	decimal numbers	3.14, -0.5, 2e3
str	text (sequence of characters)	"hello", 'A'
bool	True or False (a kind of int: True == 1)	True, False
complex	real + imaginary part	3 + 4j
NoneType	"no value"	None

```
x = 42
print(type(x), id(x) == id(42))
x = "now a string"            # dynamic typing
print(type(x))
print(type(3.14), type(True), type(2 + 3j), type(None))
a = 10
b = a                         # both names, one object
print(a is b)
```

```
<class 'int'> True
<class 'str'>
<class 'float'> <class 'bool'> <class 'complex'> <class 'NoneType'>
True
```

Type conversion (casting) changes one type into another: `int()`, `float()`, `str()`, `bool()`. Note that `input()` always returns a string, so convert it when you need a number.

```
print(int("25") + 5, int(7.9), float("3.5"), str(100) + "!")
print(bool(0), bool(""), bool([]), bool("a"), bool(7))
print(int(True), float(3), round(3.14159, 2), abs(-9))
```

```
30 7 3.5 100!
False False False True True
1 3.0 3.14 9
```

Note: `int(7.9)` truncates (7.9 becomes 7), it does not round. Falsy values: 0, 0.0, empty string/list/tuple/dict/set, None, False. Everything else is truthy.

Mutable vs immutable. Immutable objects (int, float, str, tuple, bool) cannot be changed after creation; mutable objects (list, dict, set) can be changed in place.

1.10 Operators

Arithmetic operators work on numbers:

Operator	Name	Example	Result
+ - *	add, subtract, multiply	7 * 3	21
/	true division (always float)	7 / 2	3.5
//	floor division	7 // 2	3
%	modulus (remainder)	7 % 2	1
**	power	2 ** 5	32

```
print(7 / 2, 7 // 2, 7 % 2, 2 ** 5)
print(-7 // 2, -7 % 2)       # floor division rounds DOWN
print(10 % 3, 10 % 5)        # 0 means divisible
n = 4729
print(n % 10, n // 10)       # last digit, remaining digits
```

```
3.5 3 1 32
-4 1
1 0
9 472
```

Assignment operators combine an operation with assignment: `+= -= *= /= //= %= **=`.

```
x = 10
x += 5; print(x)
x *= 2; print(x)
x //= 4; print(x)
x **= 2; print(x)
```

```
15
30
7
49
```

Relational (comparison) operators compare two values and give `True` or `False`: `== != > < >= <=`. Python also allows **chained** comparisons.

```
a, b = 5, 8
print(a == b, a != b, a < b, a >= b)
print(1 < a < 10)            # chained: 1 < a and a < 10
print("apple" < "banana")     # strings compare alphabetically
print(0.1 + 0.2 == 0.3)      # beware of float precision!
```

```
False True True False
True
True
False
```

Boolean (logical) operators combine conditions:

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

```
age, has_id = 20, True
print(age >= 18 and has_id)
print(age < 18 or has_id)
print(not has_id)
print(0 or "default", 5 and 9)   # and/or return one of the operands
print(False and print("never printed"))   # short-circuit
```

```
True
True
False
default 9
False
```

Other useful operators: membership `in` / `not in` and identity `is` / `is not`.

```
print("a" in "cat", 3 not in [1, 2, 3])
x = [1]; y = [1]
print(x == y, x is y)        # equal values, different objects
```

```
True False
True False
```

1.11 Operator Precedence

When an expression has several operators, Python evaluates them in a fixed order of **precedence** (highest first). Operators of the same level are evaluated left to right (except `**`, which is right to left). Use brackets to make the order clear.

Level	Operators
1 (highest)	( ) brackets
2	** power
3	+x -x (unary sign)
4	* / // %
5	+ - (binary)

Level	Operators
6	< <= > >= == != is in
7	not
8	and
9 (lowest)	or

```
print(2 + 3 * 4)          # 14, not 20
print((2 + 3) * 4)        # 20
print(2 ** 3 ** 2)        # 512 (right to left: 2 ** 9)
print(-3 ** 2)            # -9 (** before unary minus)
print(10 - 4 - 3)         # 3 (left to right)
print(5 > 3 and 2 > 4 or 6 > 1)   # (T and F) or T -> True
print(not 5 > 3)          # not (5 > 3) -> False
```

```
14
20
512
-9
3
True
False
```

Remember: Python is case-sensitive, uses indentation for blocks, `input()` returns a string, and `/` is true division while `//` is floor division.

1.12 Types of Errors

Error type	When it happens	Example
Syntax error	Rule of the language is broken; program will not start	print("hi" (missing bracket), if x > 5 (missing colon)
Runtime error (exception)	Program starts but fails while running	10 / 0, int("abc"), my_list[99]
Logical error	Program runs but gives a wrong answer	using + instead of *, wrong condition

```
tests = ["10 / 0", "int('abc')", "[1, 2][5]", "undefined_name", "'a' + 5", "{'a': 1}['b']"]
for t in tests:
    try:
        eval(t)
    except Exception as e:
        print(type(e).__name__, "->", e)
```

```
ZeroDivisionError -> division by zero
ValueError -> invalid literal for int() with base 10: 'abc'
IndexError -> list index out of range
NameError -> name 'undefined_name' is not defined
TypeError -> can only concatenate str (not "int") to str
KeyError -> 'b'
```

A **try / except** block lets the program handle a runtime error instead of crashing:

```
try:
    n = int(input("Enter a number: "))
    print("Half:", 100 / n)
except ValueError:
    print("That was not a number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

```
Enter a number: 0
Cannot divide by zero
```

1.13 Solved Programs

```
# Simple interest and compound interest
p, r, t = 10000, 8, 3
si = p * r * t / 100
ci = p * (1 + r / 100) ** t - p
print("SI =", si, " CI =", round(ci, 2))

# Celsius to Fahrenheit
c = 37
print(c, "C =", c * 9 / 5 + 32, "F")

# Area and circumference using math
import math
radius = 7
print(round(math.pi * radius ** 2, 2), round(2 * math.pi * radius, 2))

# Swap two numbers without a third variable
a, b = 15, 40
a, b = b, a
print(a, b)

# Convert seconds to hh:mm:ss
sec = 7384
print(f"{sec // 3600:02d}:{sec % 3600 // 60:02d}:{sec % 60:02d}")
```

```
SI = 2400.0  CI = 2597.12
37 C = 98.6 F
153.94 43.98
40 15
02:03:04
```

UNIT II — Conditional Statements and Repetition

Decision making with if / elif / else, and loops with for / while / nested loops

2.1 The if Statement

A program normally runs top to bottom. A **selection statement** lets it choose which block to run depending on a condition. The block after `if` runs only when the condition is `True`. Remember the **colon** and the **indentation**.

```
marks = 72
if marks >= 40:
    print("You passed")
print("Done")              # always runs
```

```
You passed
Done
```

2.2 if - else

```
n = 17
if n % 2 == 0:
    print(n, "is even")
else:
    print(n, "is odd")
```

```
17 is odd
```

2.3 if - elif - else (else-if ladder)

Use `elif` when there are many exclusive choices. Python checks the conditions from top to bottom and runs the **first** true block only; `else` is the fallback.

```
marks = 82
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "Fail"
print("Grade:", grade)
```

```
Grade: B
```

2.4 Nested if

An `if` inside another `if` is called a **nested** if. The inner condition is checked only when the outer one is true.

```
year = 2024
if year % 4 == 0:
    if year % 100 == 0:
        if year % 400 == 0:
            print("Leap year")
        else:
            print("Not a leap year")
    else:
        print("Leap year")
else:
    print("Not a leap year")
```

```
Leap year
```

The same leap-year test using logical operators in a single condition:

```
year = 1900
if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
    print("Leap year")
else:
    print("Not a leap year")
```

```
Not a leap year
```

2.5 More Decision Examples

```
a, b, c = 14, 29, 21
if a >= b and a >= c:
    big = a
elif b >= c:
    big = b
else:
    big = c
print("Largest:", big)
```

```
Largest: 29
```

```
a, b, c = 3, 4, 5
if a + b > c and b + c > a and a + c > b:
    if a == b == c:
        print("Equilateral")
    elif a == b or b == c or a == c:
        print("Isosceles")
    else:
        print("Scalene")
else:
    print("Not a triangle")
```

```
Scalene
```

Conditional expression (ternary): a one-line if-else that produces a value: `value_if_true if condition else value_if_false`.

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)
print("even" if 8 % 2 == 0 else "odd")
```

```
Adult
even
```

Note: Use `pass` as a placeholder for an empty block (`if x: pass`). Compare with ==, never use = inside a condition.

2.6 The for Loop and range()

A loop repeats a block of code. The `for` loop visits every item of a sequence (a range, string, list, ...). The function `range()` generates numbers and has three forms:

Form	Meaning	Example	Numbers produced
range(stop)	0 up to stop-1	range(5)	0 1 2 3 4
range(start, stop)	start up to stop-1	range(2, 6)	2 3 4 5
range(start, stop, step)	jump by step (can be negative)	range(10, 0, -3)	10 7 4 1

```
for i in range(1, 6):
    print(i, end=" ")
print()
for i in range(10, 0, -3):
    print(i, end=" ")
print()
for ch in "PYTHON":
    print(ch, end="-")
print()
for fruit in ["apple", "kiwi"]:
    print(fruit.upper())
```

```
1 2 3 4 5
10 7 4 1
P-Y-T-H-O-N-
APPLE
KIWI
```

Common patterns: accumulator (running total), counter, and finding a maximum.

```
total = 0
for n in range(1, 101):
    total += n
print("Sum 1..100 =", total)

fact = 1
for n in range(1, 6):
    fact *= n
print("5! =", fact)

count = 0
for ch in "programming":
    if ch in "aeiou":
        count += 1
print("Vowels:", count)
```

```
Sum 1..100 = 5050
5! = 120
Vowels: 3
```

2.7 The while Loop

`while` repeats as long as its condition is true. Use it when you do not know in advance how many times to repeat. The loop variable must change inside the loop, otherwise the loop never ends (an **infinite loop**).

```
n = 5
while n > 0:
    print(n, end=" ")
    n -= 1
print("Liftoff!")
```

```
5 4 3 2 1 Liftoff!
```

```
# digit sum and reversal of a number
n = 4729
s = rev = 0
while n > 0:
    d = n % 10
    s += d
    rev = rev * 10 + d
    n //= 10
print(s, rev)
```

```
22 9274
```

```
# sentinel loop: stop when the user enters 0
total = 0
x = int(input("Enter number (0 to stop): "))
while x != 0:
    total += x
    x = int(input("Enter number (0 to stop): "))
print("Total:", total)
```

```
Enter number (0 to stop): 5
Enter number (0 to stop): 12
Enter number (0 to stop): 8
Enter number (0 to stop): 0
Total: 25
```

2.8 break, continue, pass and loop-else

Keyword	Effect
break	leaves the loop immediately
continue	skips the rest of this round and goes to the next round
pass	does nothing (placeholder)
else on a loop	runs only if the loop finished normally (no break)

```
for i in range(1, 10):
    if i == 5:
        break
    print(i, end=" ")
print()
for i in range(1, 8):
    if i % 2 == 0:
        continue
    print(i, end=" ")
print()
```

```
1 2 3 4
1 3 5 7
```

```
n = 29
for d in range(2, n):
    if n % d == 0:
        print(n, "is not prime"); break
else:
    print(n, "is prime")
```

```
29 is prime
```

2.9 Nested Loops

A loop inside another loop is a **nested loop**. For every one round of the outer loop, the inner loop runs completely. Nested loops are used for tables, patterns and 2D data. For **row patterns**: the outer loop controls the rows, the inner loop controls what is printed in each row.

```
for i in range(1, 6):
    for j in range(1, i + 1):
        print("*", end=" ")
    print()
```

```
*
* *
* * *
* * * *
* * * * *
```

```
n = 4
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

```
   *
  ***
 *****
*******
```

```
for i in range(1, 4):
    for j in range(1, 6):
        print(i * j, end="\t")
    print()
```

```
1■2■3■4■5■
2■4■6■8■10■
3■6■9■12■15■
```

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end="")
    print()
```

```
1
12
123
1234
12345
```

Pattern thinking: (1) count the rows, (2) for each row decide the number of spaces, (3) the number of symbols, (4) print them, then go to the next line with `print()`.

Remember: Use `for` when the number of repetitions is known (a sequence or range) and `while` when it depends on a condition. Always make sure a `while` loop can end.

2.10 Solved Programs

```
# Multiplication table
n = 7
for i in range(1, 11):
    print(f"{n} x {i:2d} = {n * i:3d}")
```

```
7 x  1 =   7
7 x  2 =  14
7 x  3 =  21
7 x  4 =  28
7 x  5 =  35
7 x  6 =  42
7 x  7 =  49
7 x  8 =  56
7 x  9 =  63
7 x 10 =  70
```

```
# Prime numbers up to 40
for n in range(2, 41):
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            break
    else:
        print(n, end=" ")
print()

# Fibonacci series (first 10 terms)
a, b = 0, 1
for _ in range(10):
    print(a, end=" ")
    a, b = b, a + b
print()

# GCD by Euclid's method and LCM
x, y = 48, 18
p, q = x, y
while q:
    p, q = q, p % q
print("GCD =", p, " LCM =", x * y // p)
```

```
2 3 5 7 11 13 17 19 23 29 31 37
0 1 1 2 3 5 8 13 21 34
GCD = 6  LCM = 144
```

```
# Palindrome number and Armstrong number
n = 153
rev, t, s = 0, n, 0
while t > 0:
    d = t % 10
    rev = rev * 10 + d
    s += d ** len(str(n))
    t //= 10
print("Palindrome" if rev == n else "Not palindrome")
print("Armstrong" if s == n else "Not Armstrong")

# Count digits, then print the factors of a number
n = 360
print(len(str(n)), [d for d in range(1, n + 1) if n % d == 0][:8])
```

```
Not palindrome
Armstrong
3 [1, 2, 3, 4, 5, 6, 8, 9]
```

More pattern programs - study how the number of spaces and symbols changes in each row:

```
n = 4
for i in range(n, 0, -1):          # inverted triangle
    print("* " * i)
for i in range(1, n + 1):          # hollow square border rows
    print("*" * n if i in (1, n) else "*" + " " * (n - 2) + "*")
k = 1
for i in range(1, 5):              # Floyd's triangle
    for j in range(i):
        print(k, end=" "); k += 1
    print()
```

```
* * * *
* * *
* *
*
****
*  *
*  *
****
1
2 3
4 5 6
7 8 9 10
```

```
# Guessing game logic using while True and break
secret, tries = 7, 0
guesses = iter([3, 9, 7])           # pretend user input
while True:
    g = next(guesses); tries += 1
    if g < secret:
        print(g, "too low")
    elif g > secret:
        print(g, "too high")
    else:
        print(g, "correct in", tries, "tries"); break
```

```
3 too low
9 too high
7 correct in 3 tries
```

UNIT III — Functions and Strings

Writing reusable code with functions, and working with text using strings

3.1 What Is a Function?

A **function** is a named block of code that performs one task. You write it once and call it whenever needed. Functions make programs shorter, easier to read, test and fix. Python has **built-in functions** (`print`, `len`, `max`, `range`) and **user-defined functions** that you create with `def`.

```
def greet():
    print("Hello, welcome to Python!")

greet()
greet()
```

```
Hello, welcome to Python!
Hello, welcome to Python!
```

Note: `def` only *defines* the function. It runs only when you *call* it by writing its name with brackets.

3.2 Parameters, Arguments and return

Parameters are the variable names in the definition; **arguments** are the actual values passed when calling. `return` sends a value back to the caller and ends the function. A function without `return` returns `None`.

```
def add(a, b):          # a, b are parameters
    return a + b

result = add(5, 3)      # 5, 3 are arguments
print(result, add(10, 20))

def show(msg):
    print(msg)
print(show("hi"))       # show returns None
```

```
8 30
hi
None
```

```
def min_max(nums):
    return min(nums), max(nums)      # returns a tuple

lo, hi = min_max([4, 9, 1, 7])
print(lo, hi)

def is_even(n):
    return n % 2 == 0
print(is_even(10), is_even(7))
```

```
1 9
True False
```

Remember: `return` gives a value back (use it later); `print` only displays it. Code written after `return` in a function never runs.

3.3 Flow of Control with Functions

When a function is called, the program **jumps** to the function body, runs it, and then **returns** to the line right after the call. Follow the numbers in this trace:

```
def square(n):
    print("  inside square, n =", n)
    return n * n

print("1. program starts")
r = square(4)
print("2. back in main, r =", r)
print("3. total =", square(2) + r)
```

```
1. program starts
  inside square, n = 4
2. back in main, r = 16
  inside square, n = 2
3. total = 20
```

3.4 Arguments: Positional, Keyword, Default

```
def intro(name, age):
    print(name, "is", age)

intro("Riya", 19)             # positional: order matters
intro(age=20, name="Aman")    # keyword: order does not matter
```

```
Riya is 19
Aman is 20
```

Default parameters take a fallback value if the caller does not supply one. Parameters with defaults must come **after** those without.

```
def power(base, exp=2):
    return base ** exp

print(power(5))        # exp = 2
print(power(5, 3))     # exp = 3
print(power(exp=4, base=2))

def make_list(x, box=None):        # safe default for a list
    if box is None:
        box = []
    box.append(x)
    return box
print(make_list(1), make_list(2))
```

```
25
125
16
[1] [2]
```

Note: Never use a mutable object such as `box=[]` as a default value; it is created only once and shared between calls.

Variable-length arguments: `*args` collects extra positional arguments into a **tuple**; `**kwargs` collects extra keyword arguments into a **dictionary**.

```
def total(*args):
    return sum(args)
print(total(1, 2, 3), total(10, 20, 30, 40))

def profile(**kwargs):
    for k, v in kwargs.items():
        print(k, "=", v)
profile(name="Riya", city="Delhi")

nums = [3, 4, 5]
print(total(*nums))          # unpack a list into arguments
```

```
6 100
name = Riya
city = Delhi
12
```

3.5 Scope of Variables

Scope is the region where a name is visible. A variable created inside a function is **local** (exists only while the function runs). A variable created outside is **global**. Python looks for a name in this order (**LEGB**): Local, Enclosing function, Global, Built-in.

```
x = 100                # global
def show():
    y = 5              # local
    print(x, y)
show()
try:
    print(y)
except NameError as e:
    print("Error:", e)
```

```
100 5
Error: name 'y' is not defined
```

```
count = 0
def increase():
    global count       # allow changing the global variable
    count += 1
increase(); increase()
print(count)

def outer():
    n = 0
    def inner():
        nonlocal n     # change the enclosing variable
        n += 1
    inner(); inner()
    return n
print(outer())
```

```
2
2
```

Note: Assigning to a name inside a function creates a **new local** variable even if a global of the same name exists. Prefer passing values in and returning results instead of using `global`.

Mutable arguments: numbers, strings and tuples cannot be changed by a function, but a list or dictionary passed in **can** be modified in place.

```
def change(n, lst):
    n = n + 10           # only changes the local copy
    lst.append(99)       # changes the caller's list
a, nums = 5, [1, 2]
change(a, nums)
print(a, nums)
```

```
5 [1, 2, 99]
```

3.6 Lambda Functions

A **lambda** is a small anonymous (nameless) one-line function: `lambda parameters: expression`. It can have many parameters but only one expression, and returns its value automatically.

```
square = lambda n: n * n
add = lambda a, b: a + b
print(square(6), add(2, 3))
print((lambda x, y: x if x > y else y)(8, 3))

nums = [1, 2, 3, 4, 5, 6]
print(list(map(lambda n: n * 2, nums)))
print(list(filter(lambda n: n % 2 == 0, nums)))
words = ["banana", "kiwi", "apple"]
print(sorted(words, key=lambda w: len(w)))
```

```
36 5
8
[2, 4, 6, 8, 10, 12]
[2, 4, 6]
['kiwi', 'apple', 'banana']
```

`map` applies a function to every item; `filter` keeps only the items for which the function is true; `sorted()` and `max()` accept a `key` function to decide the order.

3.7 Functions as Objects

In Python a function is an **object** like any other value. You can assign it to a variable, pass it to another function, return it from a function, and store it in a list or dictionary.

```
def hello():
    return "hello"
f = hello                  # no brackets: refers to the function itself
print(f())

def apply(func, x):        # a function that takes a function
    return func(x)
print(apply(len, "python"), apply(abs, -9))

ops = {"add": lambda a, b: a + b, "mul": lambda a, b: a * b}
print(ops["add"](3, 4), ops["mul"](3, 4))

def multiplier(n):         # a function that returns a function
    def times(x):
        return x * n
    return times
double = multiplier(2)
print(double(21), hello.__name__)
```

```
hello
6 9
7 12
42 hello
```

Docstring: a string as the first statement documents the function; `help(func)` shows it.

```
def area(l, w):
    """Return the area of a rectangle."""
    return l * w
print(area.__doc__)
```

```
Return the area of a rectangle.
```

3.8 The String Type

A **string** (`str`) is an ordered, **immutable** sequence of characters. Write it in single, double or triple quotes. Triple quotes allow multi-line text.

```
a = 'single'; b = "double"
c = """line one
line two"""
print(a, b)
print(c)
print(len("Python"), type("x"))
print("It's fine", 'He said "hi"')
```

```
single double
line one
line two
6 <class 'str'>
It's fine He said "hi"
```

Escape sequences begin with a backslash: `\n` newline, `\t` tab, `\\` backslash, `\"` quote. A **raw string** (`r"..."`) ignores escapes.

```
print("A\tB\nC")
print("C:\\new\\file")
print(r"C:\new\file")
```

```
A■B
C
C:\new\file
C:\new\file
```

3.9 String Operations: Indexing, Slicing, Operators

Each character has an index: **0** for the first and **-1** for the last. Slicing `s[start:stop:step]` takes characters from start up to (but not including) stop.

```
  P   y   t   h   o   n
  0   1   2   3   4   5      (positive index)
 -6  -5  -4  -3  -2  -1      (negative index)
```

```
s = "Python"
print(s[0], s[-1], s[2])
print(s[1:4], s[:3], s[3:], s[:])
print(s[::2], s[::-1])        # every 2nd, reversed
try:
    s[0] = "J"
except TypeError as e:
    print("Error:", e)
```

```
P n t
yth Pyt hon Python
Pto nohtyP
Error: 'str' object does not support item assignment
```

```
a, b = "Good", "Day"
print(a + " " + b)            # concatenation
print(a * 3)                  # repetition
print("oo" in a, "x" not in a)
print("apple" < "banana", "a" == "A")
for ch in "abc":
    print(ch, ord(ch), end=" | ")
print(chr(72))
```

```
Good Day
GoodGoodGood
True True
True False
a 97 | b 98 | c 99 | H
```

3.10 String Methods

Strings are immutable, so a method never changes the original; it **returns a new string**.

Group	Methods (examples)
Case	upper(), lower(), title(), capitalize(), swapcase()
Remove spaces	strip(), lstrip(), rstrip()
Search	find(x) (-1 if absent), index(x) (error if absent), count(x), startswith(x), endswith(x)
Replace / split	replace(old, new), split(sep), join(list), partition(sep)
Test	isalpha(), isdigit(), isalnum(), isspace(), isupper(), islower()
Align / fill	center(w), ljust(w), rjust(w), zfill(w)

```
s = "  Hello, Python World  "
t = s.strip()
print(t.upper(), t.lower(), t.title())
print(t.find("Python"), t.find("Java"), t.count("o"))
print(t.replace("Python", "Java"))
print(t.startswith("Hello"), t.endswith("x"))
words = t.split()                  # split on whitespace
print(words, "-".join(words))
print("a,b,c".split(","))
print("123".isdigit(), "abc".isalpha(), "a1".isalnum())
print("7".zfill(3), "hi".center(8, "*"), "hi".ljust(5) + "|")
```

```
HELLO, PYTHON WORLD hello, python world Hello, Python World
7 -1 3
Hello, Java World
True False
['Hello,', 'Python', 'World'] Hello,-Python-World
['a', 'b', 'c']
True True True
007 ***hi*** hi   |
```

```
s = "madam"
print("Palindrome" if s == s[::-1] else "Not palindrome")
print(" ".join(w.capitalize() for w in "the quick fox".split()))
print(sum(1 for ch in "Programming" if ch.lower() in "aeiou"))
```

```
Palindrome
The Quick Fox
3
```

3.11 Formatted Output for Strings

There are three ways to build formatted text. **f-strings** (recommended) put expressions inside curly braces. A **format specification** follows a colon: `{value:[align][width][,][.precision][type]}`

```
name, marks, avg = "Riya", 88, 84.56789
print(f"{name} scored {marks} (avg {avg:.2f})")
print(f"{name:<8}|{marks:>5}|{avg:^10.1f}|")
print(f"{1234567:,}  {0.256:.1%}  {255:b}  {255:x}  {7:03d}")
print(f"{name!r}  {marks * 2}  {avg:08.3f}")
```

```
Riya scored 88 (avg 84.57)
Riya    |   88|   84.6   |
1,234,567  25.6%  11111111  ff  007
'Riya'  176  0084.568
```

str.format() and the older **%** operator do the same job:

```
print("{} is {} years old".format("Aman", 20))
print("{1} {0}".format("world", "hello"))
print("{n:>6.2f}".format(n=3.14159))
print("%s has %d marks and %.1f%%" % ("Neha", 90, 90.0))
print("%5d|%-5d|%05d" % (42, 42, 42))
```

```
Aman is 20 years old
hello world
  3.14
Neha has 90 marks and 90.0%
   42|42   |00042
```

Spec	Meaning	Spec	Meaning
< > ^	left, right, centre align	.2f	2 decimal places
10	minimum width 10	d / s	integer / string
,	thousands separator	%	percentage
0	pad with zeros	b / x	binary / hex

Remember: Strings are immutable sequences: index from 0, slice with `[start:stop:step]`, and use methods that return new strings. Use f-strings for formatted output.

3.12 Solved Programs on Functions and Strings

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False
    return True

def factorial(n):
    f = 1
    for i in range(2, n + 1):
        f *= i
    return f

def stats(*nums):
    return min(nums), max(nums), sum(nums) / len(nums)

print([n for n in range(1, 20) if is_prime(n)])
print(factorial(6), stats(4, 8, 15, 16, 23, 42))
```

```
[2, 3, 5, 7, 11, 13, 17, 19]
720 (4, 42, 18.0)
```

```
text = "Python is fun and Python is powerful"
words = text.split()
print(len(words), "words;", "longest:", max(words, key=len))
print("Vowels removed:", "".join(ch for ch in text if ch.lower() not in "aeiou"))
freq = {}
for ch in text.lower().replace(" ", ""):
    freq[ch] = freq.get(ch, 0) + 1
print(sorted(freq.items(), key=lambda x: (-x[1], x[0]))[:4])
print(text.swapcase()[:12], "|", text.title()[:12])
print(" ".join(w[::-1] for w in words[:3]))
```

```
7 words; longest: powerful
Vowels removed: Pythn s fn nd Pythn s pwrfl
[('n', 4), ('o', 3), ('p', 3), ('f', 2)]
pYTHON IS FU | Python Is Fu
nohtyP si nuf
```

```
def check_password(p):
    return (len(p) >= 8 and any(c.isdigit() for c in p)
            and any(c.isupper() for c in p) and any(c.islower() for c in p))

for pw in ["abc123", "Strong9pass", "alllowercase1"]:
    print(f"{pw:<15}", "OK" if check_password(pw) else "WEAK")
```

```
abc123          WEAK
Strong9pass     OK
alllowercase1   WEAK
```

UNIT IV — Lists, Tuples, Dictionaries and Sets

Python collection types: how to create, access, change and use them

4.1 What Is a List?

A **list** is an ordered, **mutable** collection written in square brackets. It can hold items of different types and allows duplicates.

```
nums = [10, 20, 30, 40]
mixed = [1, "two", 3.0, [4, 5], True]
empty = []
print(nums, len(mixed), type(empty))
print(list("abc"), list(range(5)))
```

```
[10, 20, 30, 40] 5 <class 'list'>
['a', 'b', 'c'] [0, 1, 2, 3, 4]
```

4.2 Indexing and Slicing

```
a = [10, 20, 30, 40, 50, 60]
print(a[0], a[-1], a[2])
print(a[1:4], a[:2], a[3:], a[::2], a[::-1])
a[0] = 99                    # lists are mutable
a[1:3] = [7, 8, 9]           # replace a slice
print(a)
m = [[1, 2, 3], [4, 5, 6]]   # nested list (2D)
print(m[1][2], len(m))
```

```
10 60 30
[20, 30, 40] [10, 20] [40, 50, 60] [10, 30, 50] [60, 50, 40, 30, 20, 10]
[99, 7, 8, 9, 40, 50, 60]
6 2
```

4.3 Iteration

```
a = ["red", "green", "blue"]
for x in a:
    print(x, end=" ")
print()
for i in range(len(a)):
    print(i, a[i], end=" | ")
print()
for i, x in enumerate(a, start=1):
    print(i, x, end=" | ")
print()
for x, y in zip([1, 2, 3], "abc"):
    print(x, y, end=" | ")
```

```
red green blue
0 red | 1 green | 2 blue |
1 red | 2 green | 3 blue |
1 a | 2 b | 3 c |
```

4.4 List Operators and Built-in Functions

```
a, b = [1, 2], [3, 4]
print(a + b, a * 2)
print(2 in a, 9 not in a, a == [1, 2])
n = [4, 1, 9, 3]
print(len(n), min(n), max(n), sum(n))
print(sorted(n), sorted(n, reverse=True), n)   # sorted() keeps n unchanged
print(list(reversed(n)))
print(any(x > 8 for x in n), all(x > 0 for x in n))
```

```
[1, 2, 3, 4] [1, 2, 1, 2]
True True True
4 1 9 17
[1, 3, 4, 9] [9, 4, 3, 1] [4, 1, 9, 3]
[3, 9, 1, 4]
True True
```

4.5 List Methods

Method	What it does
append(x)	add x at the end
extend(list)	add all items of another list
insert(i, x)	insert x before index i
remove(x)	delete the first x (error if absent)
pop() / pop(i)	remove and return the last / i-th item
clear()	remove everything
index(x)	position of the first x

Method	What it does
count(x)	how many times x appears
sort()	sort in place (key=, reverse=)
reverse()	reverse in place
copy()	make a (shallow) copy

```
a = [3, 1, 2]
a.append(5); a.extend([8, 9]); a.insert(1, 7)
print(a)
a.remove(7); print(a.pop(), a.pop(0), a)
a.sort(); print(a); a.reverse(); print(a)
print(a.index(5), a.count(2))
b = [1, 2, 3]
c = b           # same list, two names
d = b.copy()    # a new list
b.append(4)
print(c, d)
```

```
[3, 7, 1, 2, 5, 8, 9]
9 3 [1, 2, 5, 8]
[1, 2, 5, 8]
[8, 5, 2, 1]
1 1
[1, 2, 3, 4] [1, 2, 3]
```

List comprehension builds a list in one line: `[expression for item in iterable if condition]`.

```
print([n * n for n in range(1, 6)])
print([n for n in range(20) if n % 3 == 0])
print([w.upper() for w in ["a", "bc"]])
grid = [[i * j for j in range(1, 4)] for i in range(1, 4)]
print(grid)
```

```
[1, 4, 9, 16, 25]
[0, 3, 6, 9, 12, 15, 18]
['A', 'BC']
[[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

4.6 Lists vs Strings; split() and join()

Feature	List	String
Brackets	[1, 2, 3]	"abc"
Items	any type	characters only
Mutable	Yes (a[0] = 5 works)	No (immutable)
Indexing / slicing	Yes	Yes
Looping, in, len()	Yes	Yes
Methods	append, sort, ...	upper, split, ...

```
s = "to be or not to be"
words = s.split()           # string -> list
print(words)
print(" ".join(words))      # list -> string
print("-".join(["a", "b", "c"]), list("hey"))
csv = "10,20,30"
nums = [int(x) for x in csv.split(",")]
print(nums, sum(nums))
```

```
['to', 'be', 'or', 'not', 'to', 'be']
to be or not to be
a-b-c ['h', 'e', 'y']
[10, 20, 30] 60
```

4.7 Anagrams

Two words are **anagrams** if they use exactly the same letters the same number of times. A simple test: sort the letters of both words and compare.

```
def is_anagram(a, b):
    return sorted(a.lower()) == sorted(b.lower())

print(is_anagram("listen", "silent"))
print(is_anagram("hello", "world"))
groups = {}
for w in ["eat", "tea", "tan", "ate", "nat"]:
    groups.setdefault("".join(sorted(w)), []).append(w)
print(list(groups.values()))
```

```
True
False
[['eat', 'tea', 'ate'], ['tan', 'nat']]
```

4.8 Tuples (and Tuples from Lists)

A **tuple** is an ordered, **immutable** collection in round brackets. Use it for fixed data such as coordinates or dates. A single-item tuple needs a trailing comma.

```
t = (10, 20, 30)
single = (5,)
print(t[0], t[-1], t[1:], len(t), single, type((5)))
try:
    t[0] = 1
except TypeError as e:
    print("Error:", e)
print(t + (40,), t * 2, 20 in t)
print(t.count(20), t.index(30))
```

```
10 30 (20, 30) 3 (5,) <class 'int'>
Error: 'tuple' object does not support item assignment
(10, 20, 30, 40) (10, 20, 30, 10, 20, 30) True
1 2
```

Converting and building tuples: `tuple(list)`, `zip()` to make tuples from lists, and **unpacking**.

```
lst = [3, 1, 2]
t = tuple(lst)                  # list -> tuple
print(t, list(t))               # tuple -> list
names, marks = ["Riya", "Aman"], [88, 92]
pairs = list(zip(names, marks))   # list of tuples
print(pairs)
for n, m in pairs:              # unpacking in a loop
    print(n, m)
a, b, *rest = (1, 2, 3, 4, 5)
print(a, b, rest)
print(sorted(pairs, key=lambda p: p[1], reverse=True))
```

```
(3, 1, 2) [3, 1, 2]
[('Riya', 88), ('Aman', 92)]
Riya 88
Aman 92
1 2 [3, 4, 5]
[('Aman', 92), ('Riya', 88)]
```

	List	Tuple
Brackets	[ ]	( )
Changeable	Yes	No
Speed / memory	slower, larger	faster, smaller
Methods	many	only count, index
Can be a dict key	No	Yes

4.9 Dictionaries

A **dictionary** stores **key : value** pairs in curly braces. Keys must be unique and immutable (str, int, tuple); values can be anything. Dictionaries keep insertion order and give very fast lookup by key.

```
d = {"name": "Riya", "age": 19, "city": "Delhi"}
e = dict(a=1, b=2)
f = dict([("x", 1), ("y", 2)])
print(d, e, f)
print(len(d), type(d), {})
```

```
{'name': 'Riya', 'age': 19, 'city': 'Delhi'} {'a': 1, 'b': 2} {'x': 1, 'y': 2}
3 <class 'dict'> {}
```

4.10 Dictionary Indexing and Assignment

```
d = {"name": "Riya", "age": 19}
print(d["name"], d.get("age"), d.get("city"), d.get("city", "N/A"))
d["city"] = "Delhi"          # add a new key
d["age"] = 20                # change an existing value
print(d)
del d["city"]
print(d, "name" in d, "Riya" in d)   # 'in' checks KEYS
try:
    print(d["phone"])
except KeyError as e:
    print("KeyError:", e)
```

```
Riya 19 None N/A
{'name': 'Riya', 'age': 20, 'city': 'Delhi'}
{'name': 'Riya', 'age': 20} True False
KeyError: 'phone'
```

Method	Purpose
keys() / values() / items()	views of keys / values / (key, value) pairs
get(k, default)	safe lookup, no error

Method	Purpose
update(other)	add or change many pairs
pop(k) / popitem()	remove a key (return value) / remove the last pair
setdefault(k, v)	return d[k]; add k:v if k is missing
copy() / clear()	shallow copy / remove all
dict.fromkeys(keys, v)	new dict, same value for all keys

```
d = {"a": 1, "b": 2}
d.update({"b": 20, "c": 30})
print(d.keys(), list(d.values()))
print(d.pop("a"), d.popitem(), d)
d.setdefault("z", 0); print(d)
print(dict.fromkeys(["x", "y"], 0))
for k, v in {"p": 1, "q": 2}.items():
    print(k, "->", v)
```

```
dict_keys(['a', 'b', 'c']) [1, 20, 30]
1 ('c', 30) {'b': 20}
{'b': 20, 'z': 0}
{'x': 0, 'y': 0}
p -> 1
q -> 2
```

Counting and sorting with dictionaries (very common in exams):

```
text = "the cat and the hat and the bat"
count = {}
for w in text.split():
    count[w] = count.get(w, 0) + 1
print(count)
by_value = sorted(count.items(), key=lambda kv: (-kv[1], kv[0]))
print(by_value[:3])
print({k: v * 10 for k, v in count.items() if v > 1})
students = {"s1": {"name": "Riya", "marks": 88}, "s2": {"name": "Aman", "marks": 92}}
print(students["s2"]["name"], students["s1"]["marks"])
```

```
{'the': 3, 'cat': 1, 'and': 2, 'hat': 1, 'bat': 1}
[('the', 3), ('and', 2), ('bat', 1)]
{'the': 30, 'and': 20}
Aman 88
```

4.11 Sets

A **set** is an **unordered** collection of **unique** items in curly braces. It has no indexing, removes duplicates automatically and is very fast for membership tests. Its items must be immutable. An empty set is `set()`, because `{}` is an empty dictionary.

```
s = {3, 1, 2, 3, 1}
print(s, len(s))
print(set([1, 2, 2, 3]), set("hello"))
print(type({}), type(set()))
print(2 in s, 9 in s)
print(list(set([5, 1, 5, 2, 1])))     # remove duplicates
```

```
{1, 2, 3} 3
{1, 2, 3} {'h', 'e', 'l', 'o'}
<class 'dict'> <class 'set'>
True False
[1, 2, 5]
```

4.12 Set Methods

Method	Purpose
add(x) / update(iterable)	add one item / many items
remove(x) / discard(x)	delete x (remove raises KeyError if absent, discard does not)
pop() / clear()	remove an arbitrary item / remove all
copy()	shallow copy
issubset / issuperset / isdisjoint	comparison tests

```
s = {1, 2, 3}
s.add(4); s.update([5, 5, 6])
s.remove(1); s.discard(100)
print(s, s.pop() in {2, 3, 4, 5, 6})
print({1, 2}.issubset({1, 2, 3}), {1, 2, 3}.issuperset({3}), {1}.isdisjoint({2}))
```

```
{3, 4, 5, 6} True
True True True
```

4.13 Set Operators and Functions

Operation	Operator	Method	Meaning
Union	A | B	A.union(B)	all items of both
Intersection	A & B	A.intersection(B)	items common to both
Difference	A - B	A.difference(B)	items in A but not in B
Symmetric difference	A ^ B	A.symmetric_difference(B)	items in exactly one set
Subset / superset	A <= B / A >= B	issubset / issuperset	containment

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B, A & B)
print(A - B, B - A, A ^ B)
print(A.union(B) == A | B)
A.intersection_update({2, 3, 9})     # in-place version
print(A)
print(len(B), min(B), max(B), sum(B), sorted(B))
print({n * n for n in range(1, 6)})          # set comprehension
fs = frozenset([1, 2, 3])                    # immutable set
print(fs)
```

```
{1, 2, 3, 4, 5, 6} {3, 4}
{1, 2} {5, 6} {1, 2, 5, 6}
True
{2, 3}
4 3 6 18 [3, 4, 5, 6]
{1, 4, 9, 16, 25}
frozenset({1, 2, 3})
```

Typical uses of sets: removing duplicates, fast membership checks and finding common or different items between groups.

```
coding = {"Riya", "Aman", "Neha"}
music = {"Neha", "Kabir", "Aman"}
print("In both:", sorted(coding & music))
print("Only coding:", sorted(coding - music))
print("Either club:", len(coding | music))
```

```
In both: ['Aman', 'Neha']
Only coding: ['Riya']
Either club: 4
```

Quick Comparison of the Four Collections

Feature	List	Tuple	Set	Dictionary
Brackets	[ ]	( )	{ }	{k: v}
Ordered	Yes	Yes	No	Yes (insertion)
Changeable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Access by	index	index	no indexing (use in)	key

Remember: Pick the collection by the job: list for ordered changing data, tuple for fixed data, set for unique items and set maths, dictionary for key-to-value lookup.

4.14 Solved Programs on Collections

```
a = [4, 9, 2, 9, 7, 4, 1]
print(sorted(set(a))[-2])                    # second largest
seen, uniq = set(), []
for x in a:                                  # remove duplicates, keep order
    if x not in seen:
        seen.add(x); uniq.append(x)
print(uniq)
print([x for x in set(a) if a.count(x) > 1])   # duplicates
k = 2
print(a[k:] + a[:k])                         # rotate left by 2
```

```
7
[4, 9, 2, 7, 1]
[4, 9]
[2, 9, 7, 4, 1, 4, 9]
```

```
A = [[1, 2, 3], [4, 5, 6]]
B = [[7, 8, 9], [1, 2, 3]]
add = [[A[i][j] + B[i][j] for j in range(3)] for i in range(2)]
trans = [[A[i][j] for i in range(2)] for j in range(3)]
print(add)
print(trans)
print([sum(r) for r in A], sum(sum(r) for r in A))
```

```
[[8, 10, 12], [5, 7, 9]]
[[1, 4], [2, 5], [3, 6]]
[6, 15] 21
```

```
d = {"a": 1, "b": 2, "c": 3}
print({v: k for k, v in d.items()})                    # invert a dictionary
x, y = {"p": 1, "q": 2}, {"q": 20, "r": 30}
print({**x, **y})                                      # merge dictionaries
marks = {"Riya": 85, "Aman": 92, "Neha": 85}
top = max(marks, key=marks.get)
print(top, [n for n, m in marks.items() if m == 85])
groups = {}
for n, m in marks.items():
    groups.setdefault(m, []).append(n)
print(groups)
```

```
{1: 'a', 2: 'b', 3: 'c'}
{'p': 1, 'q': 20, 'r': 30}
Aman ['Riya', 'Neha']
{85: ['Riya', 'Neha'], 92: ['Aman']}
```

```
# sets: students who attended exactly one of two workshops
w1 = {"Riya", "Aman", "Neha", "Sara"}
w2 = {"Neha", "Kabir", "Sara"}
print(sorted(w1 ^ w2), sorted(w1 & w2), len(w1 | w2))
nums = [1, 2, 2, 3, 3, 3]
print(len(nums) - len(set(nums)), "duplicate entries")
```

```
['Aman', 'Kabir', 'Riya'] ['Neha', 'Sara'] 5
3 duplicate entries
```

PRACTICE QUESTIONS

Try these yourself after studying each unit

Unit I

1. Write an algorithm, pseudocode and flowchart to find the sum of the digits of a number.
2. Explain the four pillars of computational thinking with one example each.
3. What is the output of 2 + 3 * 4 ** 2 // 5 % 3? Evaluate step by step using precedence.
4. Differentiate between a statement and an expression, and between a keyword and an identifier.
5. Write a program to read the radius and print the area and circumference with 2 decimals.
6. Why does int(input()) work but input() + 5 fail? Show the fix.

Unit II

1. Read three sides and classify the triangle; handle invalid sides.
2. Print the Collatz sequence of a number using a while loop.
3. Print a hollow diamond, a number pyramid and Floyd's triangle.
4. Find all Armstrong numbers between 100 and 999 using nested logic.
5. Use break and loop-else to test whether a number is prime.
6. Read numbers until the user enters -1 and print the count, sum and average.

Unit III

1. Write a function that returns the grade for a given mark; call it for a list of marks.
2. Write a function with a default argument for simple interest rate.
3. Write a function using *args that returns the largest and smallest value.
4. Use lambda with map/filter/sorted to square even numbers and sort words by length.
5. Count vowels, consonants, digits and spaces in a string.
6. Print a bill using f-strings with aligned columns and 2 decimals.
7. Check whether a string is a palindrome and whether two strings are anagrams.

Unit IV

1. Find the second largest element and remove duplicates from a list without using set.
2. Read a sentence and print each word with its frequency using a dictionary.
3. Merge two dictionaries; invert a dictionary; sort a dictionary by value.
4. Given two lists, print common elements, elements only in the first, and all unique elements using sets.
5. Create a list of tuples (name, marks) with zip and print the topper.
6. Add two 3 x 3 matrices stored as nested lists and print the transpose.
7. Group a list of words into anagram groups.